LiveMeet AI — Project Overview

What Is This?

LiveMeet AI is a Windows desktop assistant that listens to meetings in real time, transcribes speech locally using OpenAI Whisper, and automatically sends the transcription to ChatGPT via a controlled Brave browser instance. It is designed to give you live AI assistance during calls without requiring any cloud-based audio processing — your voice data never leaves your machine.

Core Goal

Give anyone in a video call or meeting instant AI assistance by:

1. **Capturing** system audio (the meeting) or microphone (your voice) silently in the background
2. **Transcribing** speech locally using the Whisper model — no API keys, no audio upload
3. **Forwarding** the transcribed text to ChatGPT automatically so you have relevant AI context at all times
4. **Staying out of the way** — hotkey-driven, stealth Brave browser window that can float on top of other apps

---

How It Works

```
Audio (mic / system)


 NAudio (C# layer)
 3-second WAV chunks


 Local Flask Server
 (whisper_server.py)
 Whisper "medium" model

 transcribed text


 C# WPF App (MainWindow)
 aggregates sentences


 BraveChatController
```

```
Selenium → Brave Browser
→ chatgpt.com (Temporary Chat)
```

Tech Stack

Layer	Technology
Desktop UI	C# / WPF (.NET 7.0 Windows)
Audio capture	NAudio (WASAPI loopback + microphone)
Speech-to-text	Python Flask + OpenAI Whisper (local)
Browser automation	Selenium WebDriver + Brave
AI chat	ChatGPT web interface (no API key needed)
Hotkeys	Win32 P/Invoke (`RegisterHotKey`)
Screen capture	GDI32 P/Invoke

Key Features

- **Dual capture modes** — system audio (meeting sound) or microphone
- **Hotkey controls**
  - `Ctrl + Shift + S` — start system audio capture
  - `Ctrl + Shift + M` — start microphone capture
  - `Ctrl + Shift + P` — pause/stop capture
- **Silence detection** — RMS-based filter ignores background static
- **Sentence aggregation** — waits for natural pauses before sending to avoid fragmented messages
- **Stealth Brave browser** — masked user-agent and disabled automation flags to bypass bot detection
- **Always-on-top mode** — Brave browser window can float over all other windows
- **Screen share with app filtering** — capture your desktop while hiding specific windows (e.g. hide the assistant itself)
- **Persistent browser profile** — stays logged in to ChatGPT across sessions

---

Project Structure

```
livemeet_ai/
  whisper_server.py          # Python Flask STT server
  settings.json              # App configuration
  README.md
  PROJECT_OVERVIEW.md        # This file
```

```
LiveMeetAI.App/
   MainWindow.xaml / .cs      # Main UI + hotkey handling + transcript logic
   BraveChatController.cs     # Selenium browser automation
   AudioCaptureService.cs     # NAudio loopback + microphone capture
   LocalWhisperTranscriptionService.cs  # HTTP client to Flask server
   ScreenCaptureService.cs    # GDI32 screen capture with window filtering
   ScreenShareWindow.xaml / .cs  # Screen share UI
   HotkeyManager.cs           # Win32 hotkey registration
   FileLogger.cs              # File-based debug logging
   LiveMeetAI.App.csproj
.venv/                        # Python virtual environment
```

Configuration (`settings.json`)

Key	Purpose
`TranscriptionProvider`	Set to `"Local"` to use the Flask server
`LocalTranscriptionEndpoint`	URL of the Flask server (`http://127.0.0.1:8080/inference`)
`BravePath`	Path to the Brave browser executable
`UserDataDir`	Brave profile directory (preserves ChatGPT login)
`ChromeDriverDir`	Optional explicit path to chromedriver; leave empty for auto-resolution

---

What We Are Working On Now

Current branch: `feature/hide-feature`

Recently completed

Commit	What was done
Screen share with hidden apps	Capture the desktop while filtering out selected application windows (e.g. hide LiveMeetAI itself from the shared feed)
Always-on-top Brave window	Toggle button that keeps the Brave browser floating above all other windows

Commit	What was done
Browser attachment state tracking	Tracks whether the app attached to an existing Brave session vs. launched a new one — affects how it cleans up on exit
Improved Brave window focus	`BringBraveWindowToFront` method for reliable window focusing
Retry logic for ChromeDriver	Handles version mismatch errors on startup with automatic retry
Improved audio device selection	Prefer system-default recording device; tuned silence detection threshold
ChatGPT message send logging	Success/failure logging for every message sent to ChatGPT

Active focus

The `feature/hide-feature` branch is completing the **screen capture with window hiding** feature — allowing users to start a screen share while selectively excluding specific apps (such as the AI assistant itself or any sensitive window) from the captured frame. This is the last major feature block before the branch is ready to merge.

Running the App

1. **Start the Python transcription server:**

   ```
   .venv\Scripts\activate
   python whisper_server.py
   ```

2. **Launch the C# WPF app** (build and run in Visual Studio or `dotnet run`)

3. Make sure Brave is installed at the path in `settings.json`

4. Use hotkeys to start capture; Brave will open automatically and connect to ChatGPT